EXPERIMENT-1

PART A – COLLEGE DATABASE

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:
COLLEGE DATABASE:
STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

- **The ER diagram represents the College Database System. It is designed to store information about students, their classes, subjects, and internal assessment marks.**

1. **STUDENT Entity**
   The STUDENT entity stores the details of students studying in the college.

Attributes:

- USN (Primary Key)
- StudentName
- Username
- Address
- Phone No (shown as multi-valued attribute)
- Gender

2. **SEMSEC Entity**
   The SEMSEC entity represents Semester and Section details.

Attributes:

- SSID (Primary Key)
- SEM (Semester)
- SEC (Section)

3. **CLASS Relationship**
   The CLASS relationship connects STUDENT and SEMSEC.

- It represents that a student belongs to one semester and one section.
- The relationship is 1 : N

The relationship includes:

- USN
- SSID

4. **SUBJECT Entity**
   The SUBJECT entity stores details about subjects offered by the college.

Attributes:

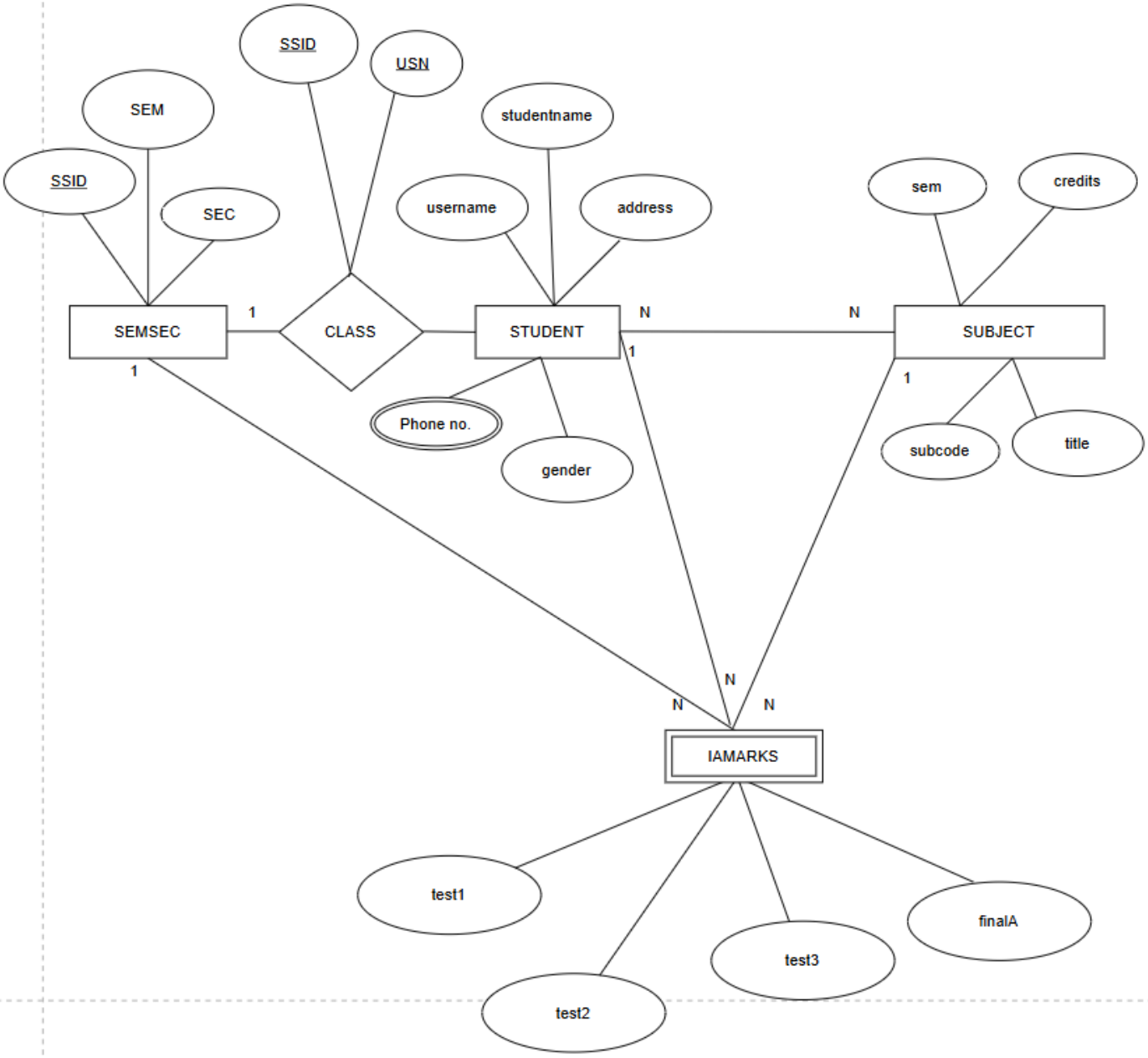
- Subcode (Primary Key)
- Title
- Semester
- Credits

5. **IAMARKS Entity**
   The IAMARKS entity stores Internal Assessment marks of students.

Attributes:

- USN
- Subcode
- SSID
- Test1
- Test2
- Test3
- FinalIA

ER DIAGRAM


SSID
USN
SEM
studentname
SSID
SEC
username
address
sem
credits
SEMSEC
1
CLASS
STUDENT
N
N
SUBJECT
1
1
1
Phone no.
gender
subcode
title
N
N
N
IAMARKS
test1
test2
test3
finalA

PART B – COMPANY DATABASE

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:
COMPANY DATABASE:
EMPLOYEE (SSN, Name, Address, Sex, Salary, SuperSSN, DNo)
DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)
DLOCATION (DNo,DLoc)
PROJECT (PNo, PName, PLocation, DNo)
WORKS_ON (SSN, PNo, Hours)

COMPANY DATABASE: The given ER diagram represents a Company Database System. It describes the structure of the organization by showing entities, their attributes, and the relationships among them.

1. DEPARTMENT Entity
The **DEPARTMENT** entity stores information about departments in the company.
Attributes:
- **DNO** (Primary Key)
- DNAME
- MgrSSN (Manager SSN)
- MgrStartDate

2. EMPLOYEE Entity
The **EMPLOYEE** entity stores employee details.
Attributes:
- **SSN** (Primary Key)
- Name
- Sex
- Address
- Salary
- SuperSSN (Supervisor SSN)
- DNO (Department Number – Foreign Key)

3. PROJECT Entity
The **PROJECT** entity contains details about company projects.
Attributes:
- **PNO** (Primary Key)
- PNAME
- PLOCATION
- DNO (Department Number – Foreign Key)

4. DLOCATION Entity
The **DLOCATION** entity represents department locations.
Attributes:
- DNO (Foreign Key)
- DLOC

5. MANAGES Relationship
The **MANAGES** relationship connects:
- EMPLOYEE
- DEPARTMENT

Cardinality:
- One department is managed by one employee.
- An employee may manage one department.

(1 : 1 relationship)

6. CONTROLS Relationship
The **CONTROLS** relationship connects:
- DEPARTMENT
- PROJECT

Cardinality:
- One department controls many projects.
- Each project is controlled by one department.

(1 : M relationship)

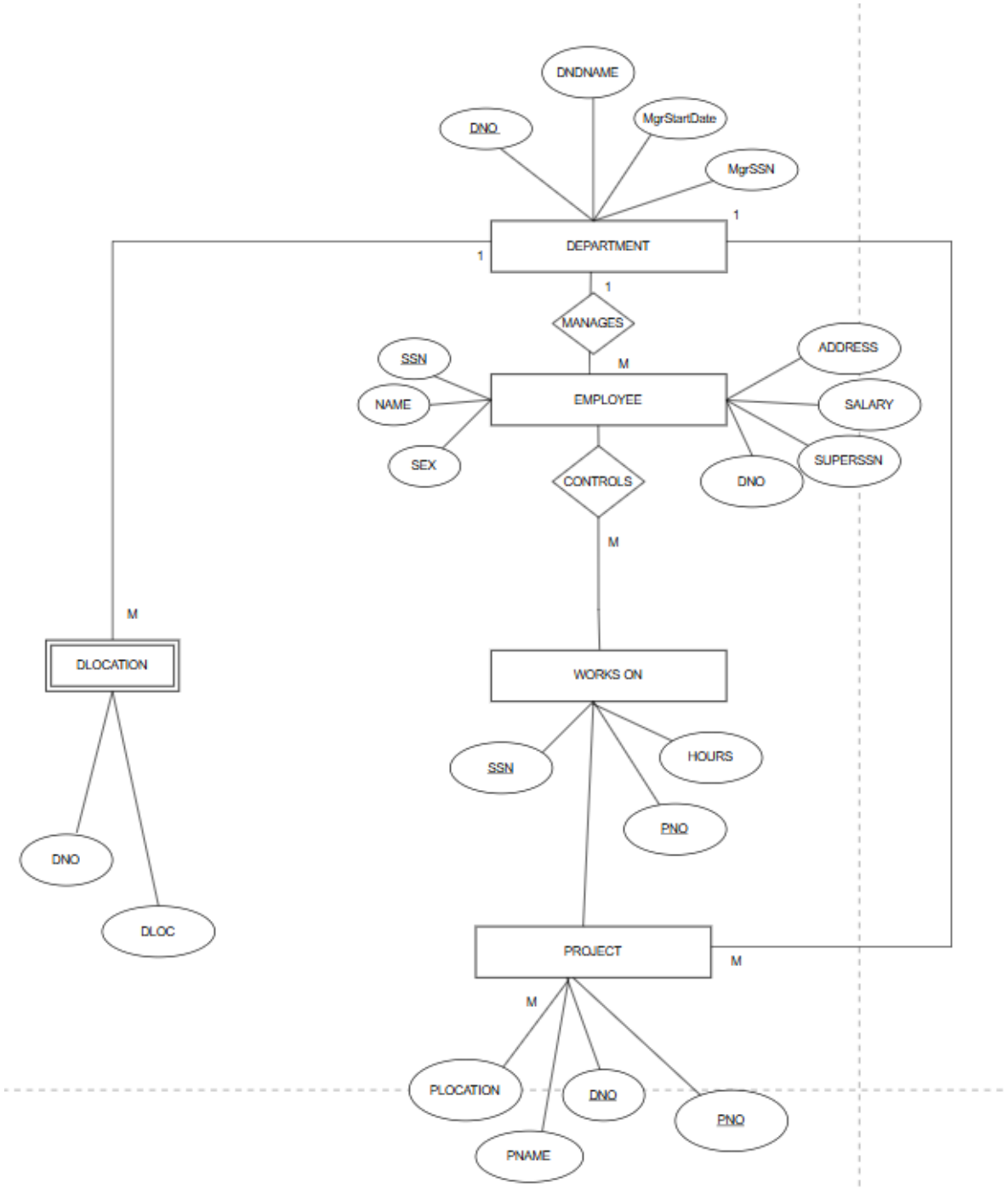
7. WORKS_ON Relationship
The **WORKS_ON** relationship connects:
- EMPLOYEE
- PROJECT

Attributes:
- SSN
- PNO
- Hours

Cardinality:
- One employee can work on many projects.
- One project can have many employees.

(M : N relationship)

ER DIAGRAM


DNDNAME
DNO
MgrStartDate
MgrSSN
DEPARTMENT
1
1
1
MANAGES
M
SSN
NAME
SEX
EMPLOYEE
ADDRESS
SALARY
SUPERSSN
DNO
CONTROLS
M
M
DLOCATION
WORKS ON
SSN
HOURS
PNO
DNO
DLOC
PROJECT
M
M
PLOCATION
DNO
PNO
PNAME

PART C – CUSTOMER SALE SCHEMA

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

This ER (Entity-Relationship) diagram represents a Sales Management System. It shows the relationship between three main entities: CUSTOMER, SALE, and ITEM.

1. CUSTOMER Entity

Attributes:

- cust_id (Primary Key)
- cust_name

2. SALE Entity

Attributes:

- bill_no (Primary Key)
- bill_date
- qty_sold

3. ITEM Entity

Attributes:

- item_id (Primary Key)
- item_name
- price

Relationships with Cardinality:

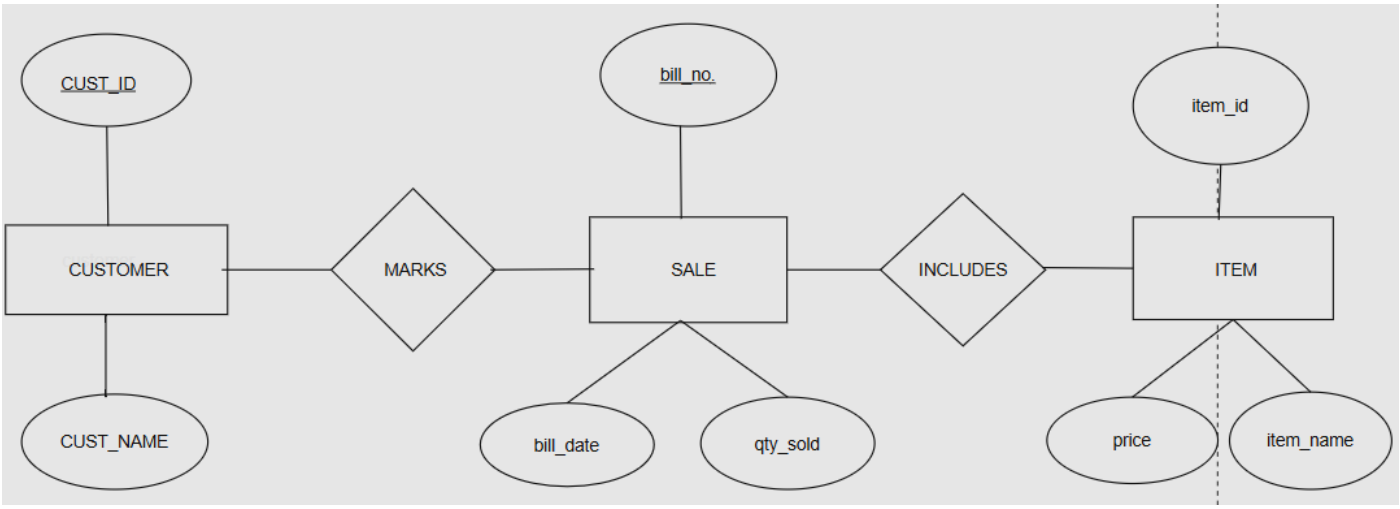
1. MAKES (1:M Relationship)

One CUSTOMER can make many SALES, but each SALE is made by only one CUSTOMER.

2. INCLUDES (M:1 Relationship)

Many SALES can include one ITEM, but each ITEM entry in this relationship refers to a single item record.

This ER diagram helps in understanding how customers, sales, and items are connected in a sales system database.



DDL Questions (Data Definition Language):

1. **Write SQL statements to create all the above relations with appropriate PRIMARY KEY and FOREIGN KEY constraints.**

```sql
CREATE DATABASE LAB1;
CREATE TABLE STUDENT (
    USN VARCHAR(20) PRIMARY KEY,
    SName VARCHAR(50),
    Address VARCHAR(100),
    Phone VARCHAR(15),
    Gender VARCHAR(10)
);
CREATE TABLE SEMSEC (
    SSID INT PRIMARY KEY,
    Sem INT,
    Sec VARCHAR(5)
);

CREATE TABLE CLASS (
    USN VARCHAR(20),
    SSID INT,
    PRIMARY KEY (USN, SSID),
    FOREIGN KEY (USN) REFERENCES STUDENT(USN),
    FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
CREATE TABLE SUBJECT (
    Subcode VARCHAR(10) PRIMARY KEY,
    Title VARCHAR(50),
    Sem INT,
    Credits INT
);
CREATE TABLE IAMARKS (
    USN VARCHAR(20),
    Subcode VARCHAR(10),
    SSID INT,
    Test1 INT,
    Test2 INT,
    Test3 INT,
    FinalIA INT,
    PRIMARY KEY (USN, Subcode, SSID),
    FOREIGN KEY (USN) REFERENCES STUDENT(USN),
    FOREIGN KEY (Subcode) REFERENCES SUBJECT(Subcode),
    FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

```sql
CREATE TABLE DEPARTMENT (
    DNo INT PRIMARY KEY,
    DName VARCHAR(50),
    MgrSSN VARCHAR(20),
    MgrStartDate DATE
);
CREATE TABLE EMPLOYEE (
    SSN VARCHAR(20) PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    Sex VARCHAR(10),
    Salary DECIMAL(10,2),
    SuperSSN VARCHAR(20),
    DNo INT,
    FOREIGN KEY (SuperSSN) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY (DNo) REFERENCES DEPARTMENT(DNo)
);
CREATE TABLE DLOCATION (
    DNo INT,
    DLoc VARCHAR(50),
    PRIMARY KEY (DNo, DLoc),
    FOREIGN KEY (DNo) REFERENCES DEPARTMENT(DNo)
);
CREATE TABLE PROJECT (
    PNo INT PRIMARY KEY,
    PName VARCHAR(50),
    PLocation VARCHAR(50),
    DNo INT,
    FOREIGN KEY (DNo) REFERENCES DEPARTMENT(DNo)
);
CREATE TABLE WORKS_ON (
    SSN VARCHAR(20),
    PNo INT,
    Hours DECIMAL(5,2),
    PRIMARY KEY (SSN, PNo),
    FOREIGN KEY (SSN) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY (PNo) REFERENCES PROJECT(PNo)
);
```

2. Add a new column Email to the STUDENT relation.

```sql
ALTER TABLE STUDENT
ADD Email VARCHAR(50);
```

3. **Apply NOT NULL constraints to Salary and Price attributes.**

```sql
ALTER TABLE EMPLOYEE
MODIFY Salary DECIMAL(10,2) NOT NULL;
ALTER TABLE Item
MODIFY price INT NOT NULL;
```

4. **Rename attribute SName to StudentName in STUDENT relation.**

```sql
ALTER TABLE STUDENT
RENAME COLUMN SName TO StudentName;
```

5. **Drop the DLOCATION relation.**

```sql
DROP TABLE DLOCATION;
```

DML Questions (Data Manipulation Language):

1. **Insert at least 10 tuples into CUSTOMER, ITEM, and SALE relations.**

```sql
CREATE TABLE Customer (
    Cust_id INT PRIMARY KEY,
    Cust_name VARCHAR(50)
);
INSERT INTO Customer VALUES
(1,'Priya'),
(2,'Rahul'),
(3,'Amit'),
(4,'Neha'),
(5,'Karan'),
(6,'Simran'),
(7,'Arjun'),
(8,'Riya'),
(9,'Vikas'),
(10,'Meena');
CREATE TABLE Item (
     item_id INT PRIMARY KEY,
     item_name VARCHAR(50),
     price INT NOT NULL
);
INSERT INTO Item VALUES
(101,'Laptop',50000),
(102,'Mouse',500),
(103,'Keyboard',800),
(104,'Headphones',1500),
(105,'Pen',20),
(106,'Notebook',100),
(107,'Bag',1200),
(108,'Phone',20000),
(109,'Charger',700),
(110,'Tablet',25000);
```

```sql
CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE,
    cust_id INT,
    item_id INT,
    qty_sold INT,
    FOREIGN KEY (cust_id) REFERENCES Customer(Cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
INSERT INTO Sale VALUES
(1,CURDATE(),1,101,1),
(2,CURDATE(),2,102,2),
(3,CURDATE(),3,104,1),
(4,'2026-02-15',5,108,1),
(5,'2026-02-14',4,105,10),
(6,'2026-02-13',5,110,1),
(7,'2026-02-12',6,103,3),
(8,'2026-02-11',7,107,1),
(9,'2026-02-10',8,109,2),
(10,CURDATE(),5,102,5);
```

2. **List all bills of the current date with customer names and item names.**

```sql
SELECT s.Bill_no, s.Bill_date, c.Cust_name, i.Item_name
FROM Sale s
JOIN Customer c ON s.Cust_id = c.Cust_id
JOIN Item i ON s.Item_id = i.Item_id
WHERE s.Bill_date = CURDATE();
```

Bill_no	Bill_date	Cust_name	Item_name
1	2026-03-02	Priya	Laptop
2	2026-03-02	Rahul	Mouse
3	2026-03-02	Amit	Headphones
10	2026-03-02	Karan	Mouse

3. **Display total bill amount using quantity and price.**

```sql
SELECT s.bill_no, c.Cust_name, i.Item_name,
       s.qty_sold, i.Price,
       (s.qty_sold * i.Price) AS Total_Amount
FROM Sale s
JOIN Customer c ON s.cust_id = c.Cust_id
JOIN Item i ON s.item_id = i.item_id;
```

Bill_no	Bill_date	Cust_name	Item_name
1	2026-03-02	Priya	Laptop
2	2026-03-02	Rahul	Mouse
3	2026-03-02	Amit	Headphones
10	2026-03-02	Karan	Mouse

4. **List customers who purchased products priced above 200.**

```
SELECT DISTINCT c.Cust_name
FROM Sale s
JOIN Customer c ON s.Cust_id = c.Cust_id
JOIN Item i ON s.Item_id = i.Item_id
WHERE i.Price > 200;
```

5. **Count the number of products purchased by each customer.**

```
SELECT c.Cust_name, COUNT(s.Item_id) AS Total_Products
FROM Sale s
JOIN Customer c ON s.Cust_id = c.Cust_id
GROUP BY c.Cust_name;
```

Cust_name	Total_Products
Priya	1
Rahul	1
Amit	1
Karan	3
Neha	1
Simran	1
Arjun	1
Riya	1

6. **List products purchased by customer having Cust_id = 5.**

```
SELECT i.Item_name, s.qty_sold
FROM Sale s
JOIN Item i ON s.Item_id = i.Item_id
WHERE s.Cust_id = 5;
```

Item_name	qty_sold
Phone	1
Tablet	1
Mouse	5

7. **Create a VIEW showing bill_no, bill_date, cust_id, item_id, price, qty,sold, amount.**

```
CREATE VIEW Bill_Details AS
SELECT s.Bill_no,
       s.Bill_date,
       s.Cust_id,
       s.Item_id,
       i.Price,
       s.qty_sold,
       (i.Price * s.qty_sold) AS Amount
FROM Sale s
JOIN Item i ON s.Item_id = i.Item_id;
```

Bill_no	Bill_date	Cust_id	Item_id	Price	qty_sold	Amount
1	2026-03-02	1	101	50000	1	50000
2	2026-03-02	2	102	500	2	1000
3	2026-03-02	3	104	1500	1	1500
4	2026-02-15	5	108	20000	1	20000
5	2026-02-14	4	105	20	10	200
6	2026-02-13	5	110	25000	1	25000
7	2026-02-12	6	103	800	3	2400
8	2026-02-11	7	107	1200	1	1200
9	2026-02-10	8	109	700	2	1400
10	2026-03-02	5	102	500	5	2500

8.Create a VIEW showing daily sales date-wise for the last one week

```
CREATE VIEW Weekly_Sales AS
SELECT Bill_date,
       SUM(i.Price * s.qty_sold) AS Daily_Total
FROM Sale s
JOIN Item i ON s.Item_id = i.Item_id
WHERE Bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY Bill_date;
SELECT * FROM Weekly_Sales;
```

Bill_date	Daily_Total
2026-03-02	55000